HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering

Web Programming

TechStore

Domain 4: An Online Electronic Store

TechStore

Instructor: NGUYEN DUC THAI, Ph.D.
Class: CC02 (Lab)
Student: Le Nguyen Nhat Minh
ID: 2352743

Ho Chi Minh City, May 7, 2026

1 Introduction

"TechStore" is a dynamic, database-driven e-commerce platform developed for the Web Programming course to fulfill the requirements of Domain D4: Electronics Store. The project simulates a real-world online retail environment where users can browse products, filter categories dynamically using AJAX, and check physical store availability. Academically, this website serves as a practical demonstration of full-stack web development principles by integrating core technologies without relying on heavy frameworks. It utilizes pure PHP for server-side logic, MySQL for data management, and a combination of HTML, CSS, and Vanilla JavaScript to ensure a responsive layout across desktop, tablet, and mobile devices. Ultimately, TechStore demonstrates a comprehensive understanding of how a modern web application operates seamlessly from the front-end to the back-end.

2 Domain-Specific Design Choices (Electronics Store)

Developing an e-commerce platform for Electronics Store topic requires distinct UI/UX and architectural considerations compared to standard retail websites. Electronic devices are high-involvement products characterized by complex specifications and high price points. Therefore, the following domain-specific design choices were implemented.

2.1 Flexible Technical Specifications Management

Unlike clothing or books, electronic products (e.g., smartphones, laptops, monitors) have vastly different and highly detailed technical specifications. To accommodate this, the database schema utilizes a *JSON* data type for the specifications column. This allows the system to flexibly store diverse attributes without creating excessive database columns. On the frontend, the Product Detail page features a dedicated, rich-text structured container to display these specifications clearly to tech-savvy consumers.

2.2 Physical Store Availability & Maps Integration

Consumers purchasing high value electronics often prefer to inspect the physical product or require immediate in-store pickup. To address this domain-specific behavior, a *"Store Availability"* section is prominently integrated into the Product Detail page. This feature not only lists the specific branches where the item is currently in stock but also provides direct Google Maps links for immediate routing, bridging the gap between online browsing and offline purchasing.

2.3 Analytical Browsing via Pagination

Purchasing technology involves careful comparison. While infinite scrolling (lazy loading) is popular in modern web design, pagination was explicitly chosen for the product catalog. Num-

bered pagination provides a stable and predictable browsing structure, allowing users to easily remember and return to specific pages when comparing multiple device models.

2.4 Precision Filtering and Search Navigation

Electronics shoppers frequently search by precise model numbers, brand names, or distinct categories (e.g., "Monitors" vs. "Smartphones"). To facilitate this, the UI employs an AJAX-powered real-time search bar and instant category filtering buttons. This allows users to narrow down technical hardware with high precision and zero page-reload latency.

3 Project System Architecture & Design Decisions

3.1 System Overview

TechStore is built on a traditional Client-Server architecture, utilizing pure PHP for server-side logic and MySQL for centralized data management. A key architectural highlight is the integration of AJAX (via Vanilla JavaScript) to perform dynamic interactions, such as real-time searching and category filtemring. This design decision significantly enhances the user experience by providing fluid updates without requiring full page reloads.

3.2 2.2. Directory Structure

To ensure organized development and maintainability, the project follows a modular file structure:

1. /app: Contains core logic, system configurations, and the PDO database connection script.
2. /database: Stores database-related assets, including the SQL schema definitions (e.g., table creation scripts) and sample data used to initialize the MySQL database.
3. /public: The main entry point of the application, hosting primary executable files (e.g., index.php, products.php) and static assets.
4. /Views: Houses UI templates, subdivided into reusable components: header.php, footer.php, and specific page layouts, etc.
5. /scripts: Contains helper utilities, including the Python migration scripts used to transfer and clean data from MongoDB to MySQL.

3.3 Layout & UI Components

The website adheres to the standard layout structure required by the project specification. The interface features a fixed header equipped with navigation menu containing primary links such

as Home, Products, Search, and Contact. Below the header, the main content body provides a flexible and responsive area specifically designed to display the product catalog, technical specifications, and store availability information. Finally, the layout is anchored by a minimalist footer that includes standard copyright notices and essential contact details.

3.4 Responsive Design Strategy

To deliver a seamless shopping experience across all devices—from desktop monitors to mobile screens—TechStore leverages a responsive Grid System and CSS Media Queries. The UI embraces a modern 'Flat Design' aesthetic, intentionally omitting resource-heavy graphical effects like glassmorphism. This ensures lightning-fast load times and smooth performance even on low-end hardware or limited network connections.

3.5 Data Pipeline Implementation

A significant design decision involved a two-stage data processing pipeline. Raw data from e-commerce sources was initially collected into MongoDB to handle its unstructured nature. Subsequently, a custom Python migration script was implemented to clean, transform, and load this data into the relational MySQL schema. This ensures data integrity and high query efficiency for the front-end application.

4 Website Layout and Structure

4.1 Global Header and Footer

The website contains a shared header and footer that appear across the main pages. The header provides brand identity, navigation links, account access, wishlist access, cart access, and an AJAX search box for quickly finding products. The footer contains supporting website information, quick links, contact information, and store-related references, helping users navigate the site consistently from any page.

4.2 Homepage - home.php

The homepage acts as the main landing page of the electronics store. It introduces the TechStore brand, highlights selected products, displays product categories, and shows a product grid for browsing. Users can explore products directly from the homepage and use category buttons to filter displayed products without needing to navigate away immediately.

4.3 Product Catalog and Dynamic Filtering - products.php

The Products page displays a comprehensive catalog loaded directly from the MySQL database. To enhance the browsing experience, it supports sorting by price and rating, alongside a pagination system to manage the extensive list of items. A key feature of this page is the AJAX-powered category filtering system, which allows users to narrow down products in real-time without a full page reload. Breadcrumb navigation (e.g., Home > Products > Category) is also implemented here to help users track their location within the store's hierarchy.

4.4 Product Detail Page and Store Integration - product_detail.php

The product detail page displays full information about a selected product. It includes the product image, name, price, rating, stock status, technical specifications, store availability, product description, and related products. This page helps users evaluate a product before adding it to the cart or saving it to the wishlist.

4.5 User Authentication and Account Management - login.php and profile.php

TechStore provides a secure authentication system that includes dedicated pages for User Registration, Login. These pages utilize clean, card-based layouts with server-side validation to ensure that all user inputs, such as email formats and password lengths, meet security standards.

4.6 Shopping Cart and Wishlist - cart.php and wishlist.php

To support the core e-commerce flow, the website includes functional Cart and Wishlist pages. The Cart page allows users to review their selected items, adjust quantities, and view the total amount before proceeding, while the Wishlist page enables users to save favorite products for future consideration. Both features leverage session-based storage and AJAX interactions to provide immediate feedback, such as toast notifications, ensuring a fluid and modern user experience.

5 Implementation Details

This project fulfills key technical requirements by focusing on four core areas: AJAX-driven interactions, efficient data pagination, robust user authentication, and foundational Search Engine Optimization (SEO). Built on a lightweight, custom MVC architecture, the application utilizes pure PHP, MySQL, HTML5, CSS3, and Vanilla JavaScript.

5.1 AJAX and Asynchronous Data Filtering

The application leverages Vanilla JavaScript and the native fetch() API to implement asynchronous AJAX communications, establishing a fluid, Single-Page Application (SPA) feel without requiring full page reloads. The core asynchronous functionalities include real-time product searching, dynamic category filtering, and session-based cart/wishlist management. All frontend requests are routed through dedicated PHP endpoints (e.g., routes/api.php handled by ApiController), which execute the necessary database queries and return structured JSON responses for dynamic DOM manipulation.

To optimize performance, the real-time search module incorporates a debounce function. This technique delays the API request until the user pauses typing, significantly reducing redundant server queries. Upon execution, a GET request retrieves matching products, allowing the frontend to instantly render a dropdown suggestion list. Similarly, the category filtering system operates asynchronously. When a category is selected, the UI displays a brief loading state while fetching the filtered dataset and corresponding pagination metadata. JavaScript then partially re-renders the product grid and pagination controls, leaving the global layout completely uninterrupted.

For transactional actions like adding items to the Cart or Wishlist, the frontend dispatches asynchronous POST requests containing product identifiers. The PHP backend processes these requests, updates the respective \$_SESSION variables, and returns a JSON status object. The frontend intercepts this response to immediately update the cart count badge and trigger non-intrusive toast notifications. This decoupled architecture—where the backend exclusively handles data and sessions while the frontend manages live DOM updates—drastically enhances system responsiveness and overall user experience.

5.2 Pagination

The pagination system is executed through the ProductController and Product model to optimize data retrieval. The controller computes an SQL offset based on the requested HTTP page parameter using the formula: offset = (currentPage - 1) * productsPerPage. Concurrently, the total number of pages is calculated via ceil(totalProducts / productsPerPage). These computed integers are dynamically bound to the MySQL query using LIMIT and OFFSET clauses, ensuring the database only fetches the exact subset of records required for the active view.

For a seamless user experience, the pagination logic preserves search and filter states by appending existing query parameters to the pagination URLs (e.g., /products\?category=2\&page=2). Within the View layer, the UI utilizes the pagination metadata to render interactive navigation controls, conditionally enabling "Previous" and "Next" buttons based on the current page boundaries.

Furthermore, this pagination architecture is fully integrated with the AJAX filtering system. When users dynamically filter categories on the frontend, the backend API returns a structured JSON payload containing both the requested product subset and the pagination metadata (total_pages, current_page). Vanilla JavaScript then asynchronously re-renders both the product grid and the corresponding pagination controls without triggering a full page reload.

5.3 User Authentication and Security

The authentication module is orchestrated through the AuthController and the User model, providing a secure and unified interface for user login, registration, and password recovery. To optimize routing, these distinct actions are processed through a single POST endpoint. The controller systematically discriminates the intended operation via a hidden form_type parameter, centralizing the authentication logic while maintaining clean MVC separation.

Security and data integrity are paramount within this module. While HTML5 attributes provide initial client-side form validation, the PHP controller acts as the definitive security layer, rigorously sanitizing and validating all inputs (e.g., email formats, password complexity, duplicate checks) to prevent malicious circumvention. Critically, the system strictly prohibits plain-text credential storage. The application employs PHP's robust password_hash() function for cryptographic credential storage, subsequently utilizing password_verify() to securely authenticate login attempts against the hashed database records.

Upon successful authentication, state persistence is managed utilizing PHP's native \$_SESSION superglobal. Essential user metadata—such as the user ID, email, and authorization role—is securely stored in the session payload. This state dictates dynamic UI rendering (e.g., conditional header navigation) and authorizes access to protected routes. Conversely, the logout mechanism explicitly clears this session data and safely destroys the active browser session, ensuring complete state termination.

5.4 SEO Implementation

To maximize organic discoverability and adhere to modern web standards, foundational Search Engine Optimization (SEO) strategies are integrated directly into the application's MVC architecture. The shared view layout is designed to dynamically inject context-specific <title> and <meta name="description"> tags passed from the controllers. This ensures that every route—from the homepage to individual product details—generates highly relevant metadata, optimizing both search engine indexing and the platform's appearance on Search Engine Results Pages (SERPs).

At the routing layer, the application uses .htaccess rewrite rules combined with the custom front controller to enforce clean URLs (e.g., /product/633 instead of /product_detail.php?id=633). This transition from query-heavy strings to semantic routing improves both user readability and algorithmic ranking factors. On the frontend, the markup strictly adheres to HTML5 semantic standards. By structuring content with meaningful tags such as <article>, <section>, and <nav>, alongside a logical heading hierarchy (<h1>, <h2>), the application provides clear structural context for both web crawlers and assistive technologies.

Furthermore, digital accessibility and image SEO are prioritized by dynamically populating alt attributes with precise product nomenclature. Finally, essential crawler directives are deployed in the public directory via a robots.txt configuration and an XML sitemap (sitemap.xml). Together, these technical implementations establish a robust SEO foundation that enhances indexability, accessibility, and the overall professionalism of the e-commerce platform.

6 Database Design Structure and Explanation

The application uses a relational database managed by MySQL. The database electronics_store is configured with the utf8mb4 character set and utf8mb4_unicode_ci collation to support international characters. The storage engine used is InnoDB, which provides support for foreign key constraints, relational integrity, and reliable transactional behavior.

The database is designed for an electronics store website. It supports product catalog management, category classification, product availability across store branches, user authentication, and customer order tracking. The schema follows relational database principles and uses primary keys, foreign keys, unique constraints, and junction tables to maintain data consistency.

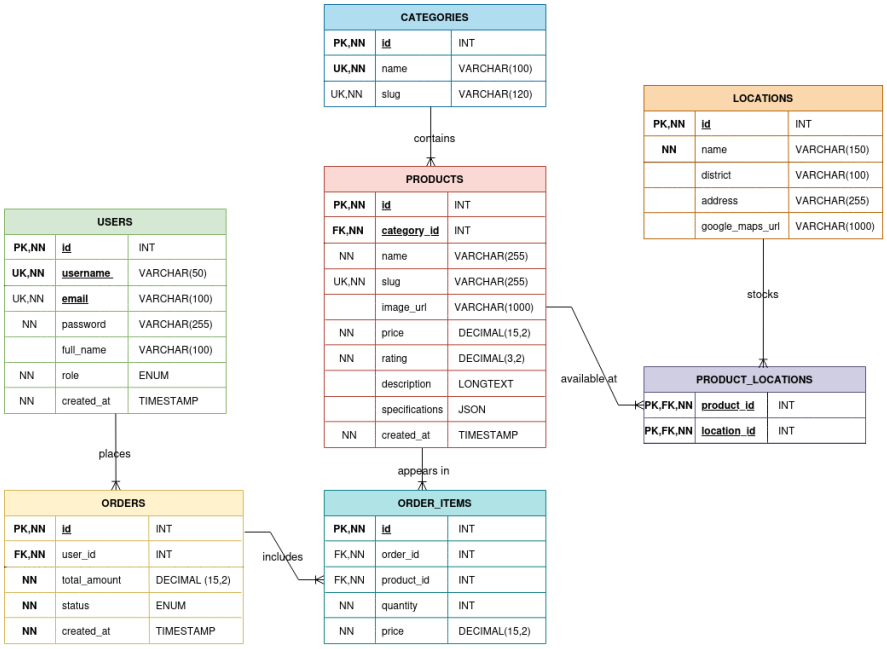

Figure 1: Our Relational Database Schema Architecture

6.1 User Authentication and Account Management - users Table

The users table stores account information for both customers and administrators. It supports the website authentication system, including registration, login, and role-based access.

The username and email columns are defined as unique fields to prevent duplicate accounts. The password column uses VARCHAR(255), which is suitable for storing hashed passwords rather than plain-text passwords. This supports secure authentication practices required by the project.

The role column uses an ENUM value with two possible roles: admin and customer. This allows the system to distinguish between normal users and administrative users. The created_at

column records when each account was created.

6.2 Product Category Management - categories Table

The categories table organizes products into groups such as phones, laptops, tablets, accessories, or other electronics-related classifications.

Each category has a unique name and slug. The slug column supports SEO-friendly URLs, such as /category/laptops or /category/smartphones. Storing slugs directly in the database helps the PHP backend retrieve category pages efficiently without generating URL strings dynamically every time.

The relationship between categories and products is One-to-Many. One category can contain many products, while each product belongs to exactly one category.

6.3 Core Product Catalog - products Table

The products table is the central table of the database. It stores all electronic items displayed on the website, including product name, slug, image URL, price, rating, description, and technical specifications.

The price column uses DECIMAL(15,2), which is appropriate for financial data because it avoids floating-point rounding errors. The rating column uses DECIMAL(3,2), allowing values such as 4.50 or 5.00.

The description column uses LONGTEXT because product descriptions may contain detailed HTML content. The specifications column uses JSON, which is suitable for storing flexible technical specifications such as RAM, storage, screen size, processor, battery capacity, or warranty information.

The category_id column is a foreign key referencing categories(id), enforcing that every product must belong to a valid category.

6.4 Branch and Product - locations and product_locations Tables

The locations table stores information about physical store branches. Each location includes the branch name, district, address, and Google Maps URL. The google_maps_url column uses VARCHAR(1000) to support long Google Maps links.

The product_locations table is a junction table that connects products with store locations. This table is necessary because the relationship between products and locations is **Many-to-Many**. One product can be available at many branches, and one branch can stock many products.

The table uses a composite primary key consisting of product_id and location_id. This prevents the same product from being assigned to the same location more than once. Both columns are

also foreign keys, referencing products(id) and locations(id).

6.5 Order Management - orders Table

The orders table stores customer purchase records. Each order belongs to one user through the user_id foreign key.

The total_amount column stores the total price of the order using DECIMAL(15,2), which is suitable for accurate financial calculations. The status column uses ENUM values including pending, processed, and cancelled. This allows the system to track the current state of each order.

The relationship between users and orders is One-to-Many. One user can place many orders, but each order belongs to only one user.

6.6 Order Details - order_items Table

The order_items table stores the individual products inside each order. This table links orders with products and records the quantity and price of each purchased item.

The price column stores the product price at the time of purchase. This is important because product prices may change later, but old orders must still preserve the original transaction value.

The relationship between orders and order_items is **One-to-Many**. One order can contain many order items. The relationship between products and order_items is also **One-to-Many** because one product can appear in many different orders.

6.7 Entity Relationships and Data Integrity

The database uses foreign keys to maintain relational integrity and prevent invalid references.

The main relationships are:

1. categories 1:N (products): A category can contain many products, but each product belongs to one category.
2. users 1:N (orders): A user can place many orders, but each order belongs to one user.
3. orders 1:N (order_items): An order can include many order items, but each order item belongs to one order.
4. products 1:N (order_items): A product can appear in many order items, but each order item references one product.
5. products M:N (locations): A product can be available at many locations, and a location can stock many products. This relationship is resolved through the product_locations junction table.

The schema uses unique constraints on fields such as username, email, category slug, and product slug to prevent duplicate data and support efficient lookup. The use of primary keys and foreign keys ensures consistency between users, products, categories, locations, and orders.

7 Challenges Faced and Lessons Learned

7.1 Challenges Faced

1. **Migration & Sanitization**: Migrating scraped NoSQL data (MongoDB) to a relational MySQL schema presented significant hurdles. Third-party HTML descriptions often contained broken inline CSS and hidden elements (e.g., `display: none;`). Resolving this required custom Python migration scripts and aggressive CSS overrides to ensure proper frontend rendering.
2. **Vanilla JS State Management**: Implementing complex asynchronous features—such as real-time search and dynamic filtering—without modern reactive frameworks (like React) was challenging. Synchronizing the DOM state with backend JSON responses required meticulous event listener management to avoid UI inconsistencies.
3. **Complex Database Design**: Designing a normalized database for diverse electronic specifications (e.g., CPU vs. battery capacity) was initially problematic due to the risk of creating highly inefficient, sparse tables.

7.2 Lessons Learned

1. **The Power of MVC Architecture**: Transitioning to a custom Model-View-Controller (MVC) architecture practically demonstrated how decoupling routing, business logic, and presentation layers drastically improves codebase maintainability and scalability.
2. **Bridging SQL and NoSQL Concepts**: Utilizing MySQL's native `JSON` datatype provided an elegant solution to the specification challenge, combining the flexibility of schemaless documents with the robust integrity of a relational database.
3. **Native Web Capabilities**: Developing SPA-like interactions reinforced the power of native browser APIs (like `fetch()`), proving that highly responsive interfaces can be achieved efficiently without bloated external libraries. Furthermore, handling scraped content highlighted the critical need for robust data sanitization and defensive programming.